

Compiled Study Notes

Machine Learning, NLP and Large Language Models

*Core concepts, definitions and theoretical explanations, merged from seven source documents.
Duplicate topics, examples, images and diagrams have been removed.*

Contents

1. Machine Learning	3
2. Machine Learning and Deep Learning Libraries	3
2.1 Scikit-learn	3
2.2 TensorFlow	3
2.3 PyTorch	3
2.4 Simple Comparison	4
3. Natural Language Processing (NLP)	4
3.1 Tokenization	4
3.2 NLP Preprocessing	4
3.3 Linguistic Analysis	5
3.4 N-grams	6
3.5 Traditional Text Representation	6
3.6 Embeddings	7
3.7 Semantic Similarity	9
4. Sequence Models	9
4.1 RNN (Recurrent Neural Network)	10
4.2 LSTM (Long Short-Term Memory)	10
4.3 GRU (Gated Recurrent Unit)	13
4.4 Comparison of Sequence Models	13
5. Large Language Models and AI Agents	13
5.1 LLM, Chatbot and Agent	13
5.2 ClawdBot / Agent Capabilities	14

1. Machine Learning

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) that enables computers to learn patterns and relationships from data and use them to make predictions or decisions without being explicitly programmed for every task.

2. Machine Learning and Deep Learning Libraries

2.1 Scikit-learn

Scikit-learn = traditional Machine Learning

Used for algorithms such as:

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- SVM
- K-Means
- PCA

It is generally the easiest place to start learning practical ML.

2.2 TensorFlow

TensorFlow = Deep Learning framework

Used to build and train neural networks such as:

- ANN
- CNN
- RNN
- LSTM
- Transformers

It is developed by Google.

2.3 PyTorch

PyTorch = Deep Learning framework

It is also used to build and train:

- ANN
- CNN
- RNN
- LSTM
- Transformers
- LLMs

It is developed by Meta and is widely used in research and modern AI development.

2.4 Simple Comparison

Library	Main use
Scikit-learn	Traditional ML
TensorFlow	Deep Learning
PyTorch	Deep Learning + AI research

3. Natural Language Processing (NLP)

Overall Progression: Text → Tokenization → Text Representation → Embeddings → Vectors → Similarity → Semantic Similarity → Applications (Semantic Search / RAG)

3.1 Tokenization

Definition: Tokenization is the process of breaking a sequence of text into smaller units called tokens. Tokens can be as small as characters or as long as words.

Key Concept: Tokenization helps machines understand and process human language by breaking it down into manageable pieces.

Applications:

- Sentiment Analysis — tokens used to judge sentiment
- Machine Translation — tokens are processed and translated into target-language tokens
- Chatbots & LLMs — input text is tokenized, then converted into token IDs (numbers)
- Text Classification — spam detection, news/topic classification, intent detection
- Named Entity Recognition (NER) — tokens help identify entities such as Person, Location, Organization
- Information Retrieval & Search — queries are tokenized to find relevant documents; also important in RAG systems
- Text Generation — language models generate text token by token, predicting the next token from previous ones
- Text Summarization — a model reads text as tokens and generates a shorter summary as tokens

3.2 NLP Preprocessing

Stop Words

Definition: Stop words are commonly used words in a language that carry little or no meaningful information for text analysis.

Important Points:

- Stop words are high-frequency, low-meaning words in text data
- Often removed during preprocessing in NLP pipelines
- Removal helps improve efficiency and model performance in some tasks
- Libraries such as NLTK and SpaCy provide built-in stopwords lists

When to remove stop words:

Task	Remove stop words?	Why?
Traditional search	Often yes	Focus on important keywords
Text classification	Sometimes	Can reduce unnecessary features
Sentiment analysis	Usually no	Words like not, never, very can change meaning
Chatbot / LLM	Usually no	Context matters
Machine translation	Usually no	Grammar and context matter
RAG / semantic search	Usually no	Embeddings benefit from the original context
TF-IDF analysis	Often yes	Can reduce common, low-information terms

Lemmatization

Definition: Lemmatization is the process of converting a word into its base or dictionary form, called its lemma.

Stemming

Definition: Stemming is the process of reducing a word to its root form by removing prefixes or suffixes. It works by cutting parts of a word, without checking whether the result is a real dictionary word.

Lemmatization vs Stemming

Lemmatization → gives a real dictionary word. Stemming → cuts word endings, result may not be a real word.

3.3 Linguistic Analysis

POS (Parts-of-Speech) Tagging

Definition: POS tagging is the process of assigning a grammatical category (part of speech) to each word in a sentence.

Key Concept: POS tagging helps an NLP system understand how a word is being used in a sentence — the same word can play different grammatical roles depending on context.

NER (Named Entity Recognition)

Definition: NER is the process of identifying real-world entities in text and assigning them a category or label.

Common NER categories: PERSON, ORG, GPE / LOCATION, DATE, MONEY, PRODUCT, EVENT.

Note: The exact labels depend on the NER model.

Applications of NER:

- Search engines — identify people, companies, places in queries
- News analysis — extract people, organizations, locations
- Document processing — extract important information
- Legal NLP — identify judges, courts, laws, cases, organizations
- Healthcare NLP — identify diseases, medicines, medical entities
- Chatbots — understand entities in user requests

- Information extraction — convert unstructured text into structured data

POS Tagging vs NER

- POS tagging — tells us the grammatical role/type of a word.
- NER — tells us what real-world entity the text refers to.

Easy Memory: POS = **What** kind of word is this? NER = **What** real-world entity is this?

3.4 N-grams

Definition: An N-gram is a sequence of N consecutive words (or tokens) taken from a text. N tells us how many consecutive tokens are included.

N	Name
1	Unigram
2	Bigram
3	Trigram
4	4-gram

Key Concept: N-grams allow a computer to understand which words commonly occur together.

Applications:

- Text prediction / autocomplete
- Spell checking
- Sentiment analysis
- Text classification
- Search engines
- Language modeling
- Machine translation

Tokenization vs N-grams

Tokenization creates individual tokens. N-grams group consecutive tokens together.

3.5 Traditional Text Representation

One-Hot / BoW / TF-IDF → sparse representation (sparse means a lot of zeros)

Word2Vec / GloVe / FastText → dense word embeddings (contain non-zero digits)

One-Hot Encoding

Definition: One-hot encoding converts each word into a vector of 0s and 1s. The word gets 1 in its own position, and 0 everywhere else.

Bag of Words (BoW)

Definition: BoW is a text representation technique that converts text into numerical vectors by counting the frequency of each word in a document, while ignoring word order.

How It Works: Raw text → Tokenization → Vocabulary → Count words → Numerical vector

Important Points:

- Called a "bag" because word order is lost.
- It is a text-to-vector / feature-extraction technique.

Limitations: BoW does not understand meaning or word order. N-grams can help address this by capturing short word sequences.

Applications:

- Text classification
- Spam detection
- Sentiment analysis
- Document clustering
- Topic extraction
- Traditional search / information retrieval

TF-IDF

Definition: TF-IDF converts text into numerical values and gives higher weight to words that are frequent in a document but rare across other documents.

Key Concept:

- **TF (Term Frequency):** how many times does this word appear in the document?
- **IDF (Inverse Document Frequency):** does this word appear in many documents or only a few?
Common words → lower score; rare, document-specific words → higher score.

Traditional Text Representation vs Embeddings

Token → ID → One-Hot → BoW / TF-IDF → Embeddings

- **One-hot** → represents token identity.
- **BoW** → represents word occurrence/counts.
- **TF-IDF** → represents word importance using frequency.
- **Embeddings** → represent learned semantic information.

3.6 Embeddings

Word Embedding

Definition: Word embedding is a technique that converts a word into a numerical vector representing information about its meaning and relationships with other words. These numbers are not manually assigned — an embedding model learns them from text.

How It Works: Text → Context → Training → Learn numerical values → Embedding vectors. The model learns from context.

Important Points:

- Comparison methods: Cosine similarity (direction/angle), Euclidean distance (straight-line distance), Manhattan distance (sums differences across dimensions), Dot product (vector alignment).
- Keep separate: the embedding model learns the vectors; a similarity/distance method compares the vectors.

Word2Vec

Definition: Word2Vec converts words into numerical vectors by learning from the words that appear around them (nearby words). Words used in similar situations get similar vectors.

Important Points:

- **CBOW** — surrounding words → predict the target word
- **Skip-gram** — target word → predict surrounding words

GloVe

Definition: GloVe (Global Vectors for Word Representation) is a word embedding technique that converts words into vectors by looking at which words appear together and how often, across a large collection of text.

FastText

Definition: FastText is a word embedding technique that represents words as vectors, but also looks at the smaller character parts inside a word — the main difference from Word2Vec.

Important Points: FastText can use shared character patterns across words to understand they are related.

Applications: Useful for rare or unseen words.

Word2Vec vs GloVe vs FastText

Method	Learns from	Special feature
Word2Vec	Nearby words	Context-based learning
GloVe	Word co-occurrence	Global co-occurrence statistics
FastText	Nearby words + character parts	Handles subwords

Word2Vec and GloVe need to have seen a word during training to give it a vector. FastText can still build a vector for an unseen word from its character pieces.

Easy Memory: Word2Vec → words + context. GloVe → words + co-occurrence. FastText → words + character parts. All three produce word embeddings.

Sentence Embedding

Definition: Sentence embedding is a technique that converts an entire sentence into a numerical vector that represents the sentence's meaning.

Word Embedding vs Sentence Embedding:

- Word embedding represents one word.
- Sentence embedding represents the whole sentence.

Applications:

- Semantic search — find sentences with similar meaning
- Question answering — match a question with relevant text
- Document similarity — compare documents
- Chatbots — understand user messages
- Sentiment analysis
- Text clustering — group similar sentences

Note: Word2Vec, GloVe, and FastText mainly create word embeddings. For sentence embeddings, models such as Sentence-BERT (SBERT) are commonly used.

Easy Memory: Word embedding → meaning of a word. Sentence embedding → meaning of a sentence. Document embedding → meaning of a document.

3.7 Semantic Similarity

Semantic Similarity

Definition: Semantic similarity is the process of measuring how similar two pieces of text are in meaning, even when they use different words.

Word Similarity vs Semantic Similarity

- Word similarity — compares individual words.
- Semantic similarity — compares entire pieces of text.

Cosine Similarity

Definition: Cosine similarity compares the direction of two vectors. Vectors pointing in nearly the same direction → high similarity; very different directions → low similarity.

Note: The exact score range and interpretation depend on the embedding model and similarity method — don't treat a particular number as a universal cutoff.

How It Works: Text 1 → Embedding → Vector 1; Text 2 → Embedding → Vector 2; Vector 1 + Vector 2 → Compare (Cosine Similarity) → Similarity Score → Estimate of meaning similarity

Key Concept: Different words can have similar vectors because embedding models learn from context — this relates to the distributional hypothesis: words that occur in similar contexts tend to have similar meanings or relationships.

Semantic Search vs Keyword Search

- Keyword search — looks for matching words.
- Semantic search — retrieves text with similar meaning, even with different words.
User Query → Embedding → Query Vector → Compare with document vectors → Most semantically similar documents

Applications: Useful when there are almost no exact matching words but the meaning is similar.

Semantic Similarity in RAG

A typical RAG (Retrieval-Augmented Generation) pipeline:

User Question → Create Embedding → Query Vector → Vector Database → Compare with chunk vectors → Find semantically similar chunks → Retrieve relevant chunks → LLM → Answer

Important Distinctions

Important Points: Semantic similarity does not mean two sentences are exactly the same — it means they are similar in meaning or topic according to the embedding model.

Three concepts to keep separate:

- **Embedding** → Represent text as numbers
- **Similarity** → Compare those numbers
- **Semantic Similarity** → Use the comparison to estimate similarity in meaning

4. Sequence Models

4.1 RNN (Recurrent Neural Network)

RNN (Recurrent Neural Network) is a type of neural network model specifically designed to process sequential data, where the order of information matters. It uses a hidden state to preserve and carry information from previous time steps as it processes the sequence.

Key terms:

- **Sequential data** → data where order matters.
- **Time step** → one position/step in the sequence.
- **Hidden state** → carries relevant information from previous time steps to the current time step.

Key Idea: Input at time t + previous hidden state → current hidden state → output

At every time step, RNN receives the current input x_t and the previous hidden state h_{t-1} , and produces:

$$h_t = f(x_t, h_{t-1})$$

Applications:

- Sentiment analysis
- Text classification
- Language modeling
- Machine translation
- Speech recognition
- Text generation

Limitations: Vanilla RNN has difficulty preserving information over many time steps because of the **vanishing gradient problem**. As the sequence becomes longer, the old information can gradually become weaker. RNN is good at short-term dependencies, but it struggles with long-term dependencies. This led to improved architectures such as LSTM and GRU.

4.2 LSTM (Long Short-Term Memory)

LSTM (Long Short-Term Memory) is a special type of **RNN** designed to handle **long-term dependencies** in sequential data.

In simple words: **LSTM is an improved form of RNN that uses gates and a separate cell state to decide what information to keep, forget, and add while processing a sequence.**

It is commonly used for:

- NLP
- Text classification
- Sentiment analysis
- Language modeling
- Speech processing
- Time-series forecasting

Main Idea: Two States

LSTM maintains:

- **Cell State C_t** — This is the **long-term memory**. It carries important information through the sequence.

- **Hidden State h_t** — This is the **short-term/current output information**. It represents what the LSTM is currently using/outputting.

A more precise description is that the cell state is the persistent memory pathway, while the hidden state is the output/state exposed at each time step.

Gates

LSTM uses **three gates** to control information: (1) Forget Gate, (2) Input Gate, (3) Output Gate.

Think of gates as **filters/controllers**. They decide: What should I forget? What new information should I store? What information should I output?

Forget Gate

The **Forget Gate** decides: **What information from the previous cell state should we forget?**

It receives the previous hidden state h_{t-1} and the current input x_t . The equation is:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Where:

- f_t = forget gate
- σ = sigmoid activation
- W_f = learnable weights
- b_f = bias

Sigmoid produces values between 0 and 1. Meaning:

- 0 → forget completely
- 1 → keep completely
- 0.5 → partially keep

Input Gate

The input gate answers: **What new information should I store in memory?** It determines which new information should be added to the cell state.

First:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

This decides **how much** new information should be accepted. Then LSTM creates candidate information:

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

So there are actually two things happening:

- **Input Gate** → How much should we add?
- **Candidate Memory** → What information should we add?

Candidate Memory

Candidate memory means: **the new information that the LSTM considers adding to its existing cell state**. It is not yet stored memory. It is a potential/new memory that LSTM has generated from the

current input and previous hidden state.

The candidate memory is calculated using $\tanh$. Here $C_t^{\sim}$ means candidate cell state/memory.

Concept	Meaning
Candidate memory $C_t^{\sim}$	New information being considered
Input gate i_t	Decides how much candidate information to add
Cell state C_t	The actual updated memory

Updating the Cell State

The cell state is updated:

$$C_t = f_t \odot C_{t-1} + i_t \odot C_t^{\sim}$$

This is one of the **most important LSTM equations**.

- **First part** $f_t \odot C_{t-1}$ means: take the old memory and decide how much of it to keep.
- **Second part** $i_t \odot C_t^{\sim}$ means: take the new candidate information and decide how much of it to add.

Therefore: **New Cell State = Old useful memory + New useful information**

Output Gate

Finally, LSTM asks: **What information should I output at this time step?** The output gate is:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Then the hidden state is:

$$h_t = o_t \odot \tanh(C_t)$$

What Does the Sigmoid Do?

The gates use **sigmoid**:

$$\sigma(x) = 1 / (1 + e^{-x})$$

It produces values between 0 and 1. Therefore it works well as a **gate**.

What Does tanh Do?

LSTM also uses $\tanh$. $\tanh$ produces values between -1 and +1. It is mainly used to:

- create candidate memory
- transform the cell state before producing the hidden state

So remember: **Sigmoid** → controls information → 0 to 1. **tanh** → transforms information → -1 to +1.

How to Know Which Information Is Important and Which Is Not?

The sigmoid function doesn't learn what is important; the neural network's weights learn patterns during training, and those learned weights cause the sigmoid gate to produce a high or low value.

4.3 GRU (Gated Recurrent Unit)

Definition: GRU is a type of RNN designed to remember important information for longer periods, similar to LSTM. It improves on basic RNNs by using gates to control information.

Key Concept: GRU decides what previous information to keep and what information to update.

How It Works — two main gates:

- **Update Gate** — decides how much old information to keep vs. how much new information to use.
- **Reset Gate** — decides how much of the previous information to ignore when processing new input.

Previous information + Current input → GRU (Update Gate + Reset Gate) → Updated information

4.4 Comparison of Sequence Models

RNN vs LSTM

Feature	Vanilla RNN	LSTM
Hidden state	Yes	Yes
Short-term information	Yes	Yes
Forget mechanism	No explicit gate	Forget gate
Input control	No explicit gate	Input gate
Output control	No explicit gate	Output gate
Vanishing-gradient problem	More severe	Reduced

RNN vs LSTM vs GRU

	RNN	LSTM	GRU
Basic sequence processing	Yes	Yes	Yes
Uses gates	No	Yes	Yes
Long-term information	Weak	Good	Good
Separate cell state	No	Yes	No
Main gates	None	3	2
Structure	Simple	More complex	Simpler than LSTM

Easy Memory: RNN → remembers. LSTM → remembers using 3 gates + cell state. GRU → remembers using 2 gates, with a simpler structure.

5. Large Language Models and AI Agents

5.1 LLM, Chatbot and Agent

LLM = Brain

Chatbot = LLM + Conversation

Agent = LLM + Tools + Decision-making + Actions (General Architecture)

ClawdBot: A **specific AI assistant/agent system** that uses an LLM, tools, memory, and integrations to communicate with users and perform tasks.

5.2 ClawdBot / Agent Capabilities

- **Execute commands/actions:** It can use tools to **execute commands or perform actions** on your behalf, rather than only generating text.
- **Long-term memory:** It can **store useful information from previous interactions** and use that information in future conversations.
- **Persistent:** **Persistent means the information/state remains stored even after the current conversation or session ends.** When you return later, the system can retrieve that stored information.

Key Difference

- **Long-term memory** = remembers information over a long period.
- **Persistence** = that information/state survives beyond the current session or restart.